MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 15 — Introduction à la POO

Classes, objets, attributs, méthodes — modéliser le monde réel

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

13 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 15 — Introduction à la POO **2**
0.1 Les concepts fondamentaux 2
0.2 Créer une classe — class 2
0.3 Le constructeur __init__ et self 3
0.4 Attributs de classe vs attributs d'instance 5
0.5 Les méthodes spéciales — Dunder methods 5
0.6 Un exemple complet — Classe Compte Bancaire 7
0.7 isinstance() et type() — Inspecter les objets 8
0.8 Ce qui m'a le plus surpris dans ce chapitre 9
0.9 Résumé du Chapitre 15 10

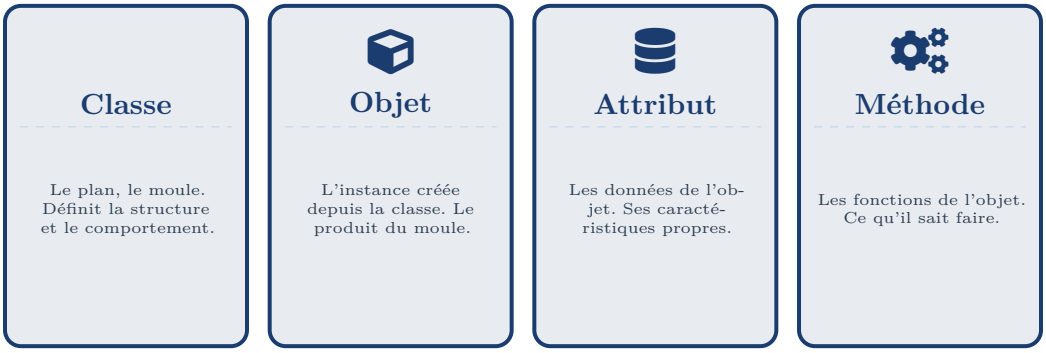
Chapitre 15 — Introduction à la POO

À retenir

La **Programmation Orientée Objet** (POO) est un paradigme qui organise le code autour d'**objets** — des entités qui combinent des **données** (attributs) et des **comportements** (méthodes). C'est la façon dont Python pense le monde : tout est objet. Comprendre la POO, c'est franchir le seuil qui sépare le programmeur débutant du développeur capable de construire des applications complexes et maintenables.

Les concepts fondamentaux

Avant d'écrire la moindre ligne, il faut comprendre les quatre piliers de la POO.



À retenir

Analogie concrète : La classe `Voiture` est le *plan de construction*. Chaque voiture créée (Alice a une Renault, Bob a une Peugeot) est un *objet* — une instance de ce plan. La couleur, la marque, la vitesse sont des *attributs*. Démarrer, freiner, accélérer sont des *méthodes*.

Créer une classe — class

Première classe Python

```python
# Définir une classe avec class
class Chien:
    """Represente un chien."""

    # Le constructeur appele a chaque creation d'objet
    def __init__(self, nom, race, age):
        # Attributs d'instance
        self.nom  = nom
        self.race = race
```

```python
        self.age  = age

    # Methode d'instance
    def aboyer(self):
        print(f"{self.nom} : Wouf Wouf !")

    def se_presenter(self):
        print(f"Je suis {self.nom}, un {self.race} de {self.age} ans.")

    def anniversaire(self):
        self.age += 1
        print(f"Joyeux anniversaire {self.nom} ! Il a maintenant {self.age} ans.")


# Instancier (creer) des objets
rex    = Chien("Rex",   "Berger Allemand", 3)
buddy  = Chien("Buddy", "Golden Retriever", 5)
luna   = Chien("Luna",  "Husky", 2)

# Appeler des methodes
rex.aboyer()
buddy.se_presenter()
luna.anniversaire()

# Acceder aux attributs
print(rex.nom)     # Rex
print(buddy.age)   # 5
print(luna.race)   # Husky
```

Le constructeur __init__ et self

Comprendre $_{init_{etself}}$

```python
class Rectangle:
    """Represente un rectangle."""

    def __init__(self, longueur, largeur):
        # self = reference vers l'objet en cours de creation
        self.longueur = longueur
        self.largeur  = largeur
        # On peut calculer des attributs derives
        self.couleur  = "blanc"   # valeur par defaut

    def aire(self):
        return self.longueur * self.largeur
```

```python
    def perimetre(self):
        return 2 * (self.longueur + self.largeur)

    def est_carre(self):
        return self.longueur == self.largeur

    def redimensionner(self, facteur):
        self.longueur *= facteur
        self.largeur  *= facteur

    def decrire(self):
        print(f"Rectangle {self.longueur}  {self.largeur}")
        print(f"  Aire       : {self.aire()} cm  ")
        print(f"  P rim tre  : {self.perimetre()} cm")
        print(f"  Carr       : {self.est_carre()}")
        print(f"  Couleur    : {self.couleur}")


# Creer des instances
r1 = Rectangle(5, 3)
r2 = Rectangle(4, 4)

r1.decrire()
r2.decrire()

# Modifier un attribut directement
r1.couleur = "rouge"
print(r1.couleur)    # rouge

# Appeler une methode qui modifie l'objet
r1.redimensionner(2)
r1.decrire()    # 10 6  maintenant
```

Ce qui m'a résisté

`self` m'a d'abord semblé redondant et bizarre. Pourquoi faut-il l'écrire partout ? En Python, `self` est la **référence explicite** vers l'objet courant. Quand on écrit `r1.aire()`, Python traduit en interne `Rectangle.aire(r1)` — `self` reçoit `r1`. Ce n'est pas une convention arbitraire : c'est la façon dont Python passe l'objet à ses propres méthodes. Une fois ce mécanisme compris, `self` devient naturel.

Attributs de classe vs attributs d'instance

Attributs de classe — partagés entre toutes les instances

```python
class Compteur:
    # Attribut de CLASSE     partage par toutes les instances
    nb_instances = 0
    espece = "Homo sapiens"

    def __init__(self, nom):
        # Attribut d'INSTANCE     propre a chaque objet
        self.nom = nom
        Compteur.nb_instances += 1   # modifier l'attribut de classe

    def se_presenter(self):
        print(f"Je suis {self.nom}. Nous sommes {Compteur.nb_instances}.")


p1 = Compteur("Alice")
p2 = Compteur("Bob")
p3 = Compteur("Charlie")

p1.se_presenter()   # Je suis Alice. Nous sommes 3.
p3.se_presenter()   # Je suis Charlie. Nous sommes 3.

# Acc der a l'attribut de classe
print(Compteur.nb_instances)   # 3
print(Compteur.espece)         # Homo sapiens
print(p1.espece)               # Homo sapiens (h rite de la classe)

# Modifier via l'instance cree un attribut d'INSTANCE
p1.espece = "Robot"     # cree p1.espece local
print(p1.espece)        # Robot
print(p2.espece)        # Homo sapiens (inchange)
print(Compteur.espece) # Homo sapiens (inchange)
```

Les méthodes spéciales — Dunder methods

Les méthodes **dunder** (double underscore) permettent de définir le comportement des objets avec les opérateurs Python.

Méthodes $_{str,repr,len,eq}$

```python
class Livre:
    def __init__(self, titre, auteur, pages):
        self.titre  = titre
```

```python
        self.auteur = auteur
        self.pages  = pages
        self.page_actuelle = 0

    # Representation lisible pour l'utilisateur
    def __str__(self):
        return f'"{self.titre}" par {self.auteur}'

    # Representation pour les developpeurs
    def __repr__(self):
        return f'Livre(titre="{self.titre}", auteur="{self.auteur}", pages={self.pages})'

    # len(livre)
    def __len__(self):
        return self.pages

    # livre1 == livre2
    def __eq__(self, autre):
        return self.titre == autre.titre and self.auteur == autre.auteur

    # livre1 < livre2 (par nombre de pages)
    def __lt__(self, autre):
        return self.pages < autre.pages

    # Permet d'utiliser + entre livres
    def __add__(self, autre):
        titre_combine = f"{self.titre} & {autre.titre}"
        return Livre(titre_combine, "Collectif", self.pages + autre.pages)


l1 = Livre("Python Expert", "Alice", 450)
l2 = Livre("Data Science", "Bob", 380)
l3 = Livre("Python Expert", "Alice", 450)

print(l1)           # "Python Expert" par Alice
print(repr(l1))     # Livre(titre="Python Expert", ...)
print(len(l1))      # 450
print(l1 == l3)     # True
print(l1 == l2)     # False
print(l1 < l2)      # False (450 > 380)

combo = l1 + l2
print(combo)        # "Python Expert & Data Science" par Collectif
print(len(combo))   # 830
```

À retenir

Les méthodes dunder permettent à nos objets de se comporter comme des types Python natifs. `__str__` est appelé par `print()` et `str()`. `__repr__` est appelé dans le shell interactif. `__len__` est appelé par `len()`. Ces méthodes sont la clé pour rendre nos classes **pythoniques**.

Un exemple complet — Classe Compte Bancaire

Classe ComplèteCompteBancaire

```python
class CompteBancaire:
    """Simule un compte bancaire simple."""

    taux_interet = 0.02   # 2%     attribut de classe

    def __init__(self, titulaire, solde_initial=0):
        self.titulaire  = titulaire
        self.__solde    = solde_initial   # attribut "prive"
        self.historique = []
        self._enregistrer(f"Ouverture du compte     Solde : {solde_initial}   ")

    def _enregistrer(self, operation):
        """Enregistre une operation dans l'historique."""
        from datetime import datetime
        horodatage = datetime.now().strftime("%d/%m/%Y %H:%M")
        self.historique.append(f"[{horodatage}] {operation}")

    def deposer(self, montant):
        if montant <= 0:
            raise ValueError("Le montant doit etre positif.")
        self.__solde += montant
        self._enregistrer(f"Depot +{montant}        Solde : {self.__solde}   ")
        print(f" Depot de {montant}    effectue.")

    def retirer(self, montant):
        if montant <= 0:
            raise ValueError("Le montant doit etre positif.")
        if montant > self.__solde:
            raise ValueError("Solde insuffisant.")
        self.__solde -= montant
        self._enregistrer(f"Retrait -{montant}        Solde : {self.__solde}   ")
        print(f" Retrait de {montant}    effectue.")

    def appliquer_interets(self):
        interets = self.__solde * CompteBancaire.taux_interet
```

```python
        self.__solde += interets
        self._enregistrer(f"Interets +{interets:.2f}
           Solde : {self.__solde:.2f}   ")

    @property
    def solde(self):
        return self.__solde

    def afficher_historique(self):
        print(f"\  n     Historique     {self.titulaire}")
        for op in self.historique:
            print(f"  {op}")

    def __str__(self):
        return f"Compte de {self.titulaire}     Solde : {self.
           __solde:.2f}   "

    def __repr__(self):
        return f'CompteBancaire("{self.titulaire}", {self.
           __solde})'


# Utilisation
compte = CompteBancaire("Alice", 1000)
compte.deposer(500)
compte.retirer(200)
compte.appliquer_interets()
print(compte)
compte.afficher_historique()

# Acces securise au solde via property
print(f"\nSolde actuel : {compte.solde:.2f}   ")
```

isinstance() et type() — Inspecter les objets

Inspecter les objets

```python
class Animal:
    def __init__(self, nom):
        self.nom = nom

class Chien(Animal):
    def aboyer(self):
        return "Wouf !"

rex = Chien("Rex")
nombre = 42
texte = "Python"
```

```python

# type()      type exact
print(type(rex))       # <class '__main__.Chien'>
print(type(nombre))    # <class 'int'>
print(type(texte))     # <class 'str'>

# isinstance()      verifie si objet est instance d'une classe
print(isinstance(rex, Chien))     # True
print(isinstance(rex, Animal))    # True  (h ritage !)
print(isinstance(rex, str))       # False

print(isinstance(nombre, int))    # True
print(isinstance(nombre, (int, float)))  # True

# hasattr()      verifier si un attribut existe
print(hasattr(rex, "nom"))       # True
print(hasattr(rex, "voler"))     # False

# getattr()      acc der a un attribut par son nom
print(getattr(rex, "nom"))                # Rex
print(getattr(rex, "age", "Inconnu"))     # Inconnu (defaut)
```

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. `self` n'est pas magique — c'est juste une convention. On pourrait l'appeler `this`, `moi`, ou `objet` — Python accepterait. Mais la convention universelle est `self`. Comprendre que Python passe automatiquement l'objet comme premier argument de chaque méthode m'a tout éclairé. Ce n'est pas du tout de la magie — c'est une mécanique transparente et cohérente.

2. Les méthodes dunder — Python parle avec nos objets. Écrire `__str__` et voir `print(mon_objet)` afficher quelque chose de beau — c'est un moment fort. Réaliser que `len()`, `+`, `==`, `<` sont tous des appels à des méthodes dunder m'a montré que Python est cohérent jusqu'au bout. Tout ce qui semble être de la magie est en réalité des protocoles bien définis.

3. La différence entre attribut de classe et d'instance. Un attribut de classe est partagé par **toutes** les instances. Mais dès qu'on le modifie via une instance, Python crée un attribut d'instance local — la classe reste intacte. Ce comportement m'a piégé : je modifiais un attribut de classe via une instance en croyant affecter toutes les instances. Il faut toujours modifier les attributs de classe via `NomClasse.attribut`.

Résumé du Chapitre 15

Ce que j'ai appris dans ce chapitre :

- La POO organise le code en **objets** qui combinent données et comportements.
- `class NomClasse:` définit une classe — le plan de construction.
- `__init__(self, ...)` — le constructeur, appelé à chaque instanciation.
- `self` — référence vers l'objet courant, premier paramètre de toute méthode.
- **Attributs d'instance** vs **attributs de classe** — portées différentes.
- **Méthodes dunder** : `__str__`, `__repr__`, `__len__`, `__eq__`, `__lt__`.
- `isinstance()` — vérifier le type d'un objet avec gestion de l'héritage.
- `hasattr()`, `getattr()` — inspecter les objets dynamiquement.

*"La POO ne change pas ce que le programme fait. Elle change **comment** on pense le programme — et ça change tout."*

KETOTSA AMÉVI CLAUDE — Mon apprentissage Python | Chapitre 15 sur 34